

Tirocinio Tafuro

La prima parte del lavoro consiste nella creazione di un'intelligenza artificiale, allenata tramite random forest con il dataset CIC-IDS2018¹.

Il progetto è diviso in due file:

- **train_model.py**: È il file che contiene tutta la logica per l'addestramento del modello e la valutazione delle metriche
- **misc.py**: Contiene varie funzioni utili alla costruzione di un dataset ed alla pulizia dei dati contenuti in esso

Il dataset

Il dataset è stato originariamente creato dall'Università del New Brunswick per analizzare attacchi DDoS, ed è il risultato della cattura del traffico rete di un IDS, per una decina di giorni di marzo 2018. Esso si compone di 80 colonne per circa 8 milioni di righe.

Tra le colonne più importanti troviamo:

- Dst Port (Destination port)
- Protocol
- Flow Duration
- Tot Fwd Pkts² (Total forward packets)
- Tot Bwd Pkts (Total backward packets)
- Label

Il traffico viene identificato come malevolo o benigno in base al valore contenuto nella colonna **Label**, che possono essere:

¹ <https://www.kaggle.com/datasets/solarmainframe/ids-intrusion-csv/data>

² Per Fwd Pkt si intendono pacchetti che viaggiano dal client al server, mentre i Bwd Pkt viaggiano dal server al client.

```
===== DATASET VALUES ==  
Label  
Benign  
DDOS attack-HOIC  
DoS attacks-Hulk  
Bot  
FTP-BruteForce  
SSH-Bruteforce  
Infiltration  
DoS attacks-SlowHTTPTest  
DoS attacks-GoldenEye  
DoS attacks-Slowloris  
DDOS attack-LOIC-UDP  
Brute Force -Web  
Brute Force -XSS  
SQL Injection  
Label  
=====
```

Inoltre, il dataset è diviso in molteplici file CSV, uno per ogni giorno di cattura dei pacchetti.

Pre-processing

Data la grande mole di entries “sporche” presenti nel dataset, una prima fase di pulizia si è resa necessaria.

Come primo step per la pulizia del dataset, sono state rimosse le entries che presentavano campi non validi (con valori NaN o infiniti) e si è iniziata la prima fase della feature importance, che è stata svolta “manualmente”, rimuovendo a priori la colonna “Timestamp” dal dataset, poiché considerata inutile.

Va, inoltre, specificato che, data la grandezza del dataset, la parte del programma che gestisce la pulizia e la costruzione del dataset stesso, lo divide in piccoli chunk e lavora su di essi, per alleggerire il carico alla memoria.

Costruzione del dataset

La fase di costruzione del dataset ha previsto l'unione dei vari file del dataset in un unico grande file CSV.

In un'ulteriore fase della costruzione è stato effettuato un downsampling dei campioni benigni selezionandone un sottoinsieme, in modo che il numero di campioni benigni sia uguale al numero campioni malevoli. Questo perché il dataset presenta di per sé molti più campioni benigni che maligni, e questo porta ad un

peggioramento delle performance dell'IA quando deve rilevare le classi meno rappresentate.

I valori possibili della colonna "Label" sono:

```
===== DATASET VALUES =====  
Label  
Benign  
DDOS attack-HOIC  
DoS attacks-Hulk  
Bot  
FTP-BruteForce  
SSH-Bruteforce  
Infiltration  
DoS attacks-SlowHTTPTest  
DoS attacks-GoldenEye  
DoS attacks-Slowloris  
DDOS attack-LOIC-UDP  
Brute Force -Web  
Brute Force -XSS  
SQL Injection  
Label  
=====
```

Per semplicità, il valore "Benign" è stato trasformato in 0, mentre tutti i valori che non sono "Benign" sono stati trasformati in 1 (come visto nella funzione `clean_dataset`).

Funzione 'load_and_prepare':

Questa è la funzione che si occupa selezionare solo una parte dei campioni benigni, in modo che il suo numero sia uguale a quello dei campioni maligni (2.169.002 campioni per classe).

Addestramento del modello

Il modello viene addestrato nella funzione `'train_model'`.

Lo splitting train/test è stato fatto prendendo in considerazione le scelte di altri programmatori (ad esempio [questo](#)).

Per quanto riguarda la scelta degli iperparametri del modello, sono state utilizzate le funzioni `GridSearchCV` e `RandomizedSearchCV` su un subset ridotto del dataset originale.

La `RandomizedSearchCV` sembra aver dato i risultati migliori in un tempo minore (circa 30 minuti contro i 46 della `GridSearchCV`). Gli iperparametri migliori sono quindi:

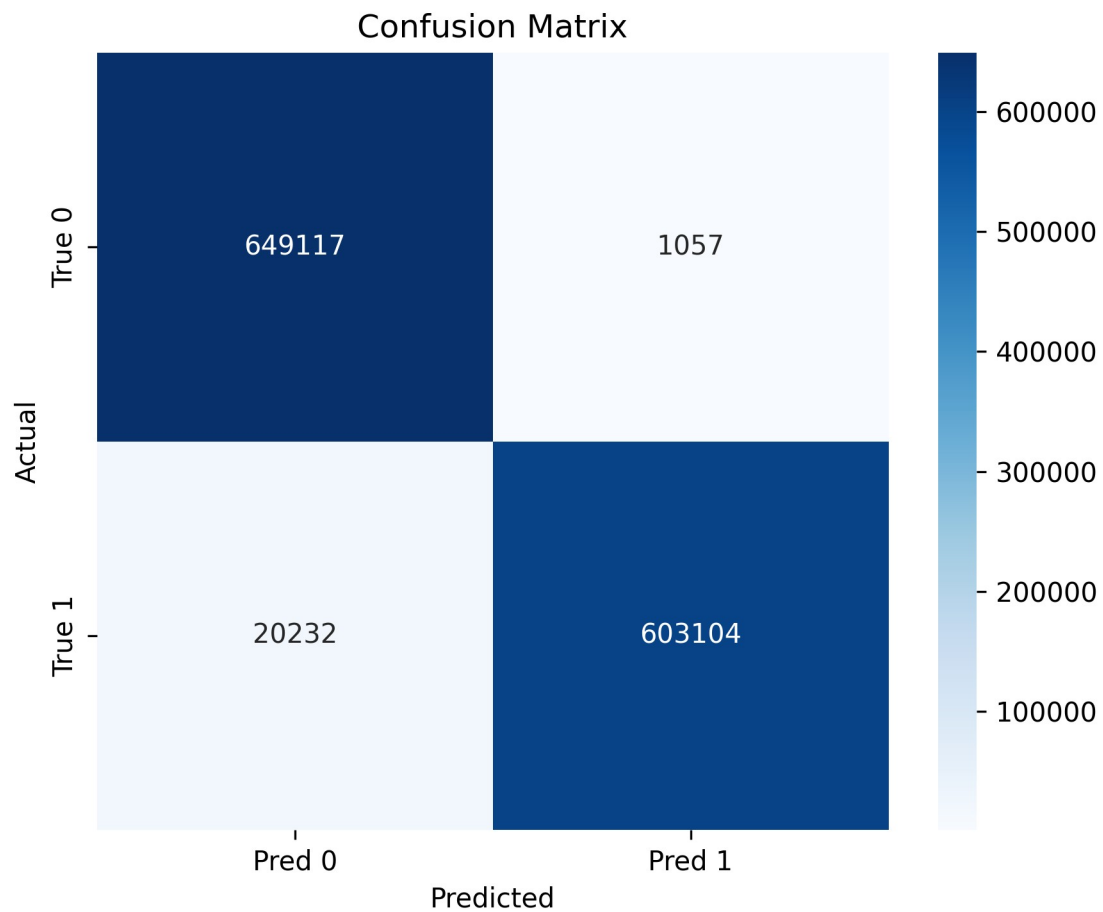
- n_estimators: 300,
- min_samples_split: 5,
- min_samples_leaf: 1,
- max_depth: 20,
- criterion: entropy

La top 10 delle combinazioni migliori è:

	mean_test_score	param_n_estimators	param_max_depth	param_min_samples_leaf
0	0.981902	300	20	1
3	0.981861	300	20	2
7	0.981763	100	30	2
8	0.981350	300	30	1
6	0.981263	100	20	4
2	0.980939	200	10	1
9	0.980924	300	10	2
1	0.980916	200	10	2
4	0.980865	300	10	2
5	0.980807	200	10	4

Valutazione del risultato

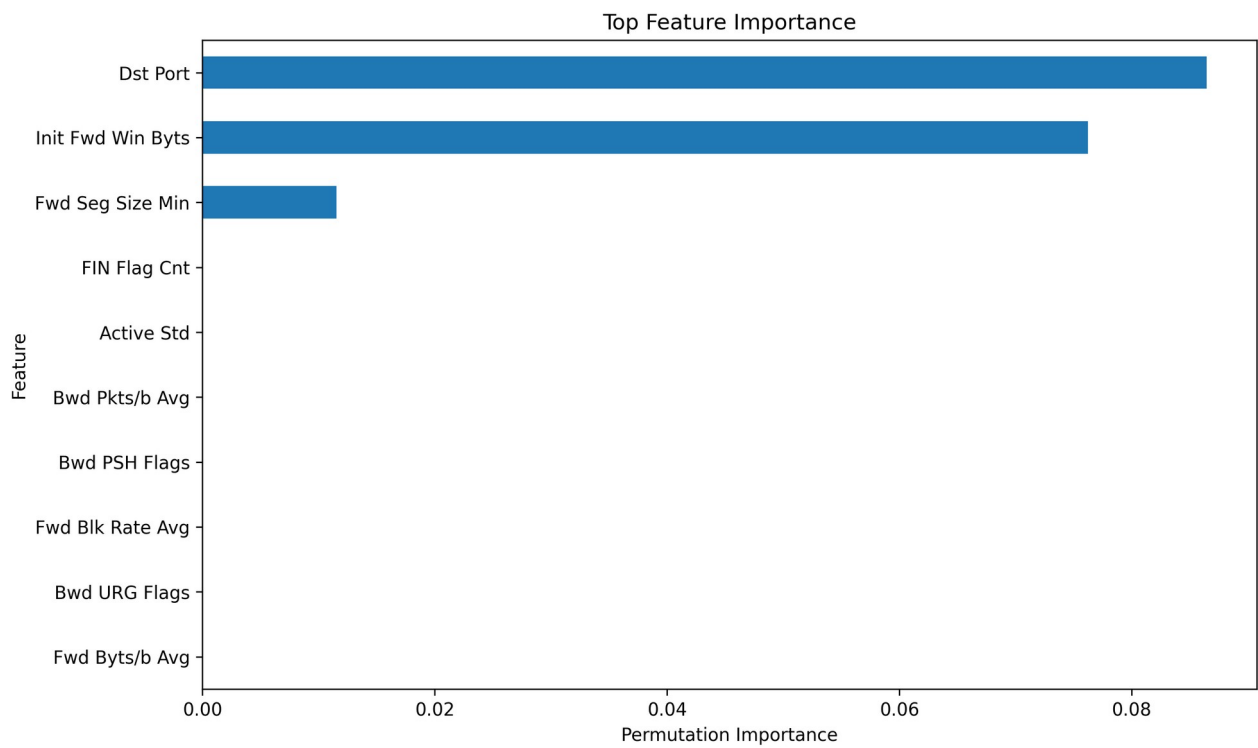
Prima di stampare i risultati sono stati eseguiti vari esperimenti sul threshold di decisione del modello (funzione *'evaluate'*) per raggiungere un valore di recall accettabile.



Funzione *'feature_importance'*:

La funzione mostra le dieci feature più importanti per il modello.

Il risultato è:



Questo grafico motra le features più importanti, cioè quelle che il modello va ad utilizzare più spesso quando deve classificare un campione.